

Project Documentation

Project Name : WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

College Name : Government College of Engineering
Erode

Team Lead : Karthick R

Team Members :

Karthick B

Mahathma E

Sanjai Harish S

Gajin S

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

User Story:

WhatNext Vision Motors is developing a Salesforce CRM solution to enhance customer experience and improve operational efficiency in the vehicle ordering process. The project focuses on automating vehicle orders by identifying the nearest dealer based on the customer's address and reducing the manual effort involved in dealer assignment.

The system verifies vehicle stock availability before processing an order and prevents unavailable vehicles from being confirmed. When the required vehicle is out of stock, the order remains in **Pending** status, while an available vehicle can be processed and updated to **Confirmed** status.

The solution also uses **Batch Apex** to periodically process pending orders when vehicle stock becomes available. In addition, the project provides test-drive management with automated email reminders, helping customers receive timely notifications about their scheduled test drives.

By combining Salesforce CRM, automation, Apex, triggers, and Batch Apex, the system improves order accuracy, reduces repetitive manual work, and provides a more organized vehicle ordering experience.

1. INTRODUCTION

1.1 Project Overview

WhatNext Vision Motors is developing a Salesforce CRM solution to modernize its vehicle ordering and dealership management activities. The system brings vehicle, dealer, customer, order, test drive, and service information into a centralized Salesforce environment.

The project automates important operations such as **nearest dealer assignment, vehicle stock validation, pending order processing, test drive management, and email reminders**. This reduces dependency on manual monitoring and helps maintain accurate information throughout the vehicle ordering process.

The system also uses Salesforce automation and Apex-based processing to ensure that vehicle orders are handled according to stock availability and business requirements. WhatNext Vision Motor

1.2 Purpose :

The purpose of this project is to streamline **vehicle ordering, stock management, dealer assignment, and customer engagement** through Salesforce CRM. The system is designed to minimize manual activities, reduce order-related errors, provide timely order information, and improve overall operational efficiency while offering a smooth customer experience.

2. IDEATION PHASE

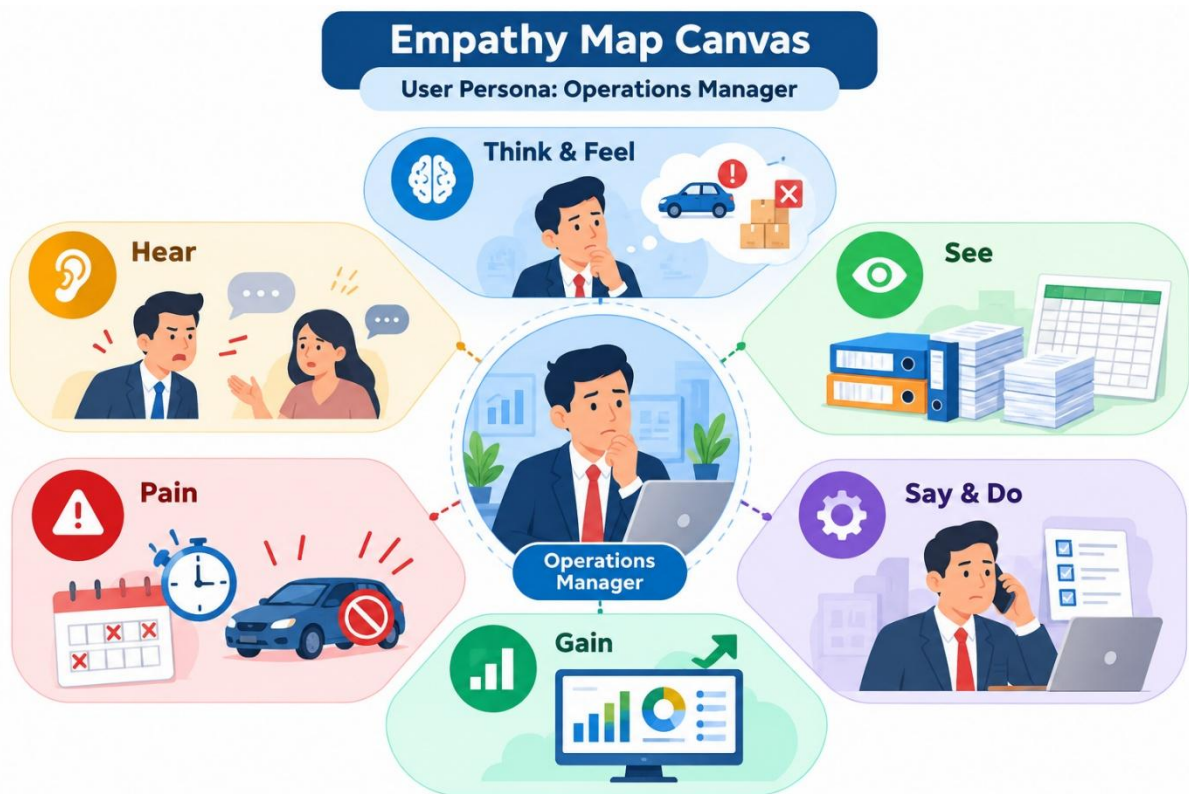
Problem Statement	PS-1: Operations Manager	PS-2: Customer
I am	Operations Manager	Customer
I'm trying to	Handle vehicle orders and monitor stock	Place a vehicle order
But	Vehicle stock information is managed manually	Vehicle availability is not clearly visible
Because	Ordering activities are handled through disconnected processes	Order information and updates may take time
Which makes me feel	Stressed	Uncertain

Problem Statement	PS-1: Operations Manager	PS-2: Customer
I am	Operations Manager 	Customer 
I'm trying to	Handle vehicle orders and monitor stock 	Place a vehicle order 
But	Vehicle stock information is managed manually 	Vehicle availability is not clearly visible 
Because	Ordering activities are handled through disconnected processes 	Order information and updates may take time 
Which makes me feel	Stressed 	Uncertain 

2.2 Empathy Map Canvas

User Persona: Operations Manager

- **Think & Feel:** Concerned about wrong orders & stock errors
- **Hear:** Dealer and customer complaints
- **See:** Registers & spreadsheets
- **Say & Do:** Manual follow-ups
- **Pain:** Delays & cancellations
- **Gain:** Automation & dashboards



2.3 Brainstorming & Idea Prioritization

Ideas Considered

- Manual register management
- Spreadsheet-based tracking
- Independent standalone portal
- Salesforce CRM

Final Selection

Salesforce CRM selected due to:

- Automation
- Scalability
- Real-time visibility



3.REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages:

Inquiry → Vehicle Selection → Order Placement → Dealer Assignment → Stock Validation → Test Drive → Confirmation → Reporting



3.2 Solution Requirements

Functional Requirements

Requirement ID	Functional Requirement
FR-1	Manage vehicle information and availability
FR-2	Maintain dealer information and details
FR-3	Create and manage customer vehicle orders
FR-4	Verify vehicle stock before order confirmation
FR-5	Automatically assign the appropriate dealer
FR-6	Schedule and manage customer test drives
FR-7	Generate reports and dashboards for monitoring

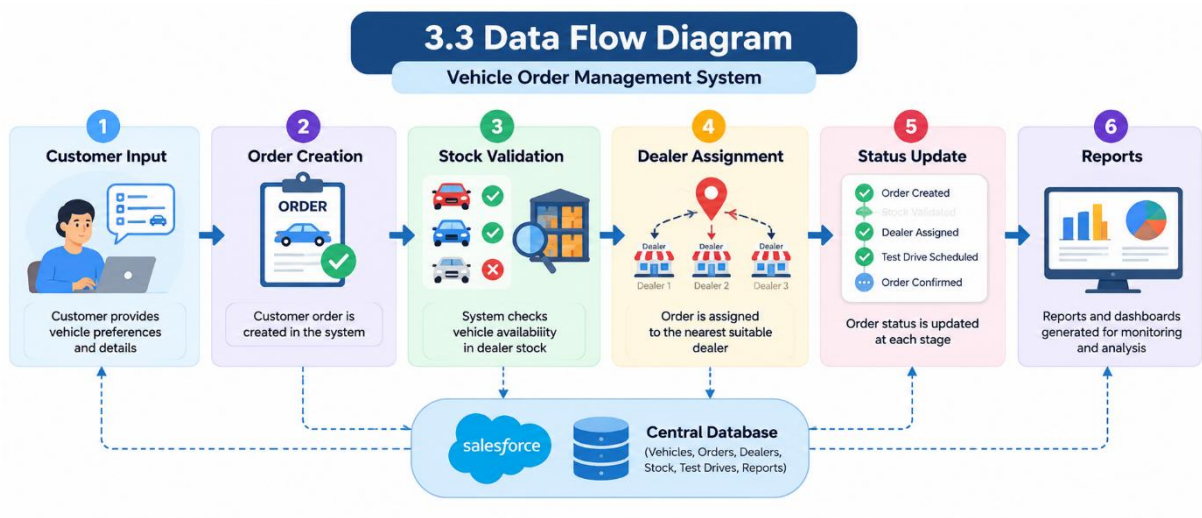
Non-Functional Requirements

Category	Requirement Description
Usability	Provide an easy-to-navigate Lightning UI
Security	Apply role-based access control
Reliability	Ensure accurate and consistent automation
Performance	Support fast vehicle order processing
Availability	Provide continuous cloud-based access
Scalability	Support operations across multiple dealers

3.3 Data Flow Diagram

Flow:

Customer Input → Order Creation → Stock Validation → Dealer Assignment → Status Update → Reports



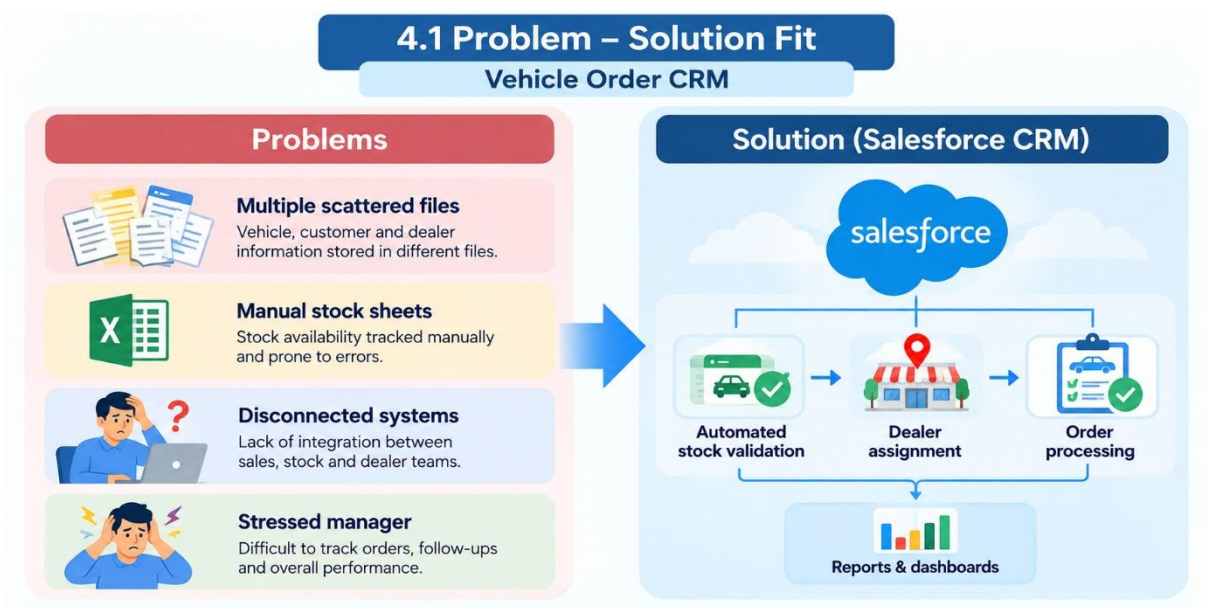
3.4 Technology Stack

Layer	Technology
User Interface	Salesforce Lightning
Business Logic	Salesforce Flows
Data Management	Salesforce Objects
Automation	Validation Rules and Apex Triggers
Processing	Batch Apex and Scheduled Apex
Reporting	Salesforce Reports and Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The proposed Salesforce solution addresses fragmented vehicle-ordering operations by integrating **stock validation, dealer assignment, and order processing** into a centralized system. This reduces manual activities, improves information flow, and supports more efficient vehicle order management.



4.2 Proposed Solution

Functional Requirements

Parameter	Description
Problem	Manual vehicle ordering
Solution	Salesforce CRM
Innovation	Automated dealer assignment
Impact	Faster service
Business Model	Dealer network
Scalability	Multi-location support

4.3 Solution Architecture

Components

- Vehicle Object
- Dealer Object
- Customer Object
- Order Object
- Test Drive Object
- Service Request Object
- Salesforce Flows
- Apex Triggers
- Batch Jobs
- Reports

- Dashboards



5. PROJECT PLANNING & SCHEDULING

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Developer Setup	USN-1	As a system administrator, I want to create and configure a Salesforce developer account so that I can build and deploy the vehicle order management system.	3	High	Sanjai Harish S

Sprint-2	Data Modeling	USN-2	As an admin, I want to create Vehicle, Dealer, Customer, Order, and Service Request objects with proper relationships so that all business data is stored and managed systematically.	5	High	Karthick R
Sprint-2	Data Modeling	USN-3	As a staff member, I want dedicated tabs and a Lightning App for all modules so that I can easily navigate and manage vehicle orders and dealer operations.	5	High	Karthick R
Sprint-3	Automation	USN-4	As a system, I want to validate stock availability, mandatory fields, and order inputs so that invalid or incomplete vehicle orders are prevented.	3	High	Karthick B
Sprint-3	Automation	USN-5	As a sales coordinator, I want automated flows to assign the nearest dealer and update order status so that processing becomes faster and requires minimal manual effort.	3	High	Karthick B
Sprint-4	Apex	USN-6	As a system, I want Apex triggers to automatically perform actions such as order confirmation and dealer assignment so that critical processes occur instantly and accurately.	5	High	Mahathma E
Sprint-4	Apex	USN-7	As an inventory manager, I want batch jobs to update vehicle stock levels automatically so that inventory data remains accurate and up to date.	5	High	Gajin S
Sprint-5	Reports	USN-8	As a manager, I want reports showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.	4	High	
Sprint-6	Dashboards	USN-9	As a manager, I want dashboards showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.	4	High	Sanjai Harish S

Project Tracker , Velocity & Burndown Chart :

Sprint	Total Story Points	Duration	Sprint Start Date	<i>Sprint End Date (Planned)</i>	<i>Story Points Completed (as on Planned End Date)</i>	<i>Sprint Release Date (Actual)</i>
Sprint-1	3	1 day	28/9/2026	28/9/2026	3	28/9/2026
Sprint-2	10	1 day	29/9/2026	29/9/2026	10	29/9/2026
Sprint-3	6	1 day	30/9/2026	30/9/2026	6	30/9/2026
Sprint-4	10	1 day	1/10/2026	1/10/2026	10	1/10/2026
Sprint-5	4	1/2 day	2/10/2026	2/10/2026	4	2/10/2026
Sprint-6	4	1/2 day	2/10/2026	2/10/2026	4	2/10/2026

6. Project Development Phase:

What you'll learn:

1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

Requirements :

1. Salesforce CRM Implementation

- Store and manage vehicle details, stock availability, and dealer information in Salesforce.
- Track customer orders, test drives, and service requests efficiently.
- Maintain customer and vehicle information in a centralized system.
- Automate the process of assigning orders to the nearest dealer based on customer location.

2. Process Automation

- Prevent orders from being confirmed when the selected vehicle is out of stock.
- Automatically assign the appropriate dealer to the customer order.
- Send automated email reminders for scheduled test drives.
- Automatically update pending orders when vehicle stock becomes available.

3. Apex and Trigger

- Implement Apex triggers to enforce business rules such as stock validation and order processing.
- Use trigger handler classes to keep the business logic modular and maintainable.
- Process vehicle orders based on stock availability.

4. Batch Jobs

- Develop a Batch Apex job to periodically check vehicle stock levels.
- Process pending vehicle orders based on current stock availability.
- Update the order status from **Pending** to **Confirmed** when stock becomes available.
- Schedule the batch process to run automatically.

What you'll learn :

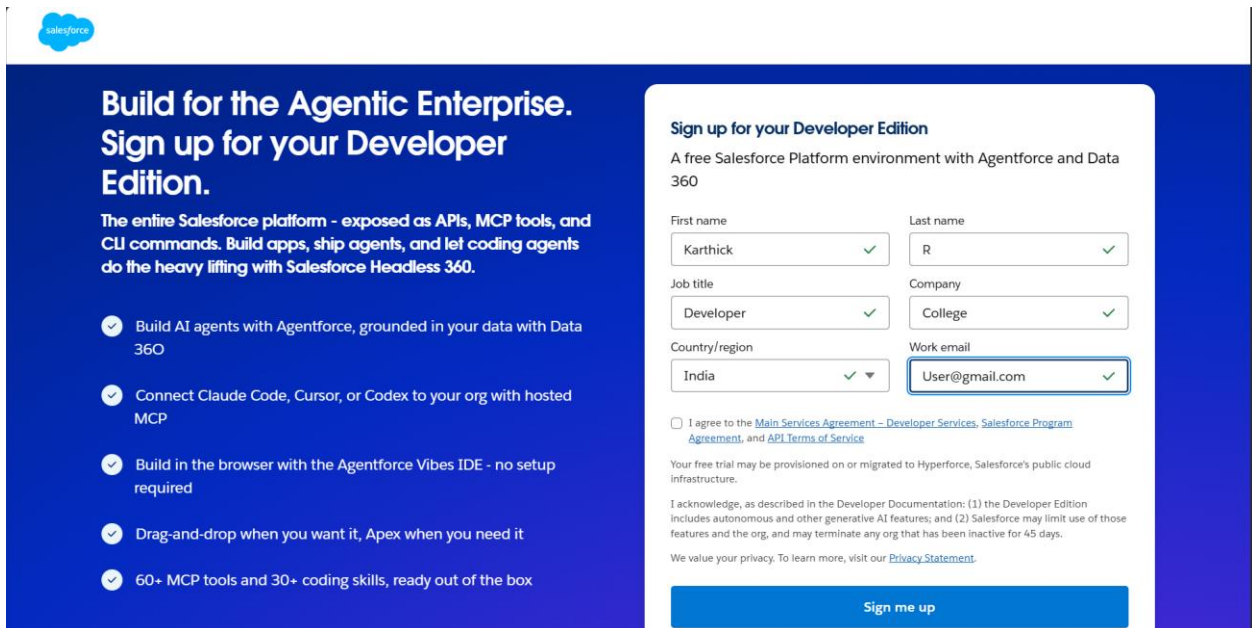
- Salesforce CRM and configuration
- Custom objects and relationships
- Custom fields and tabs
- Salesforce Flow automation
- Apex, Triggers, and Batch Apex
- Scheduling and testing automation
- Vehicle order processing
- Test drive management and reminders

MileStone 1 :

Activity 1: Create a Salesforce Developer Account

Creating a developer org in salesforce.

1. Go to <https://developer.salesforce.com/signup>
2. On the sign up form, enter the following details :



The screenshot shows the Salesforce Developer Edition sign-up page. On the left, there's a blue banner with the text "Build for the Agentic Enterprise. Sign up for your Developer Edition." and a list of features: "Build AI agents with Agentforce, grounded in your data with Data 360", "Connect Claude Code, Cursor, or Codex to your org with hosted MCP", "Build in the browser with the Agentforce Vibes IDE - no setup required", "Drag-and-drop when you want it, Apex when you need it", and "60+ MCP tools and 30+ coding skills, ready out of the box". On the right, there's a white form titled "Sign up for your Developer Edition" with the subtitle "A free Salesforce Platform environment with Agentforce and Data 360". The form contains several input fields: "First name" (Karthick), "Last name" (R), "Job title" (Developer), "Company" (College), "Country/region" (India), and "Work email" (User@gmail.com). There are checkboxes for "I agree to the Main Services Agreement – Developer Services, Salesforce Program Agreement, and API Terms of Service" and "I acknowledge, as described in the Developer Documentation: (1) the Developer Edition includes autonomous and other generative AI features; and (2) Salesforce may limit use of those features and the org, and may terminate any org that has been inactive for 45 days." A "Sign me up" button is at the bottom right.

- 1) First name & Last name
- 2) Email
- 3) Role : Developer
- 4) Company : College Name
- 5) County : India
- 6) Postal Code : pin code
- 7) Username : should be a combination of your name and company

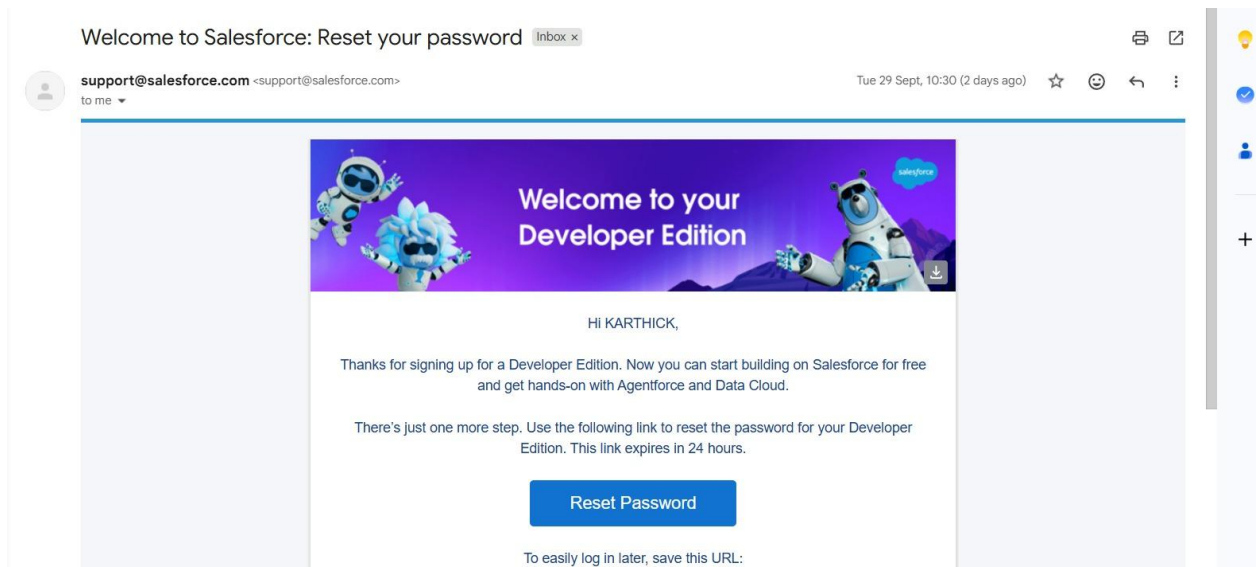
This need not be an actual email id, you can give anything in the format :

username@organization.com

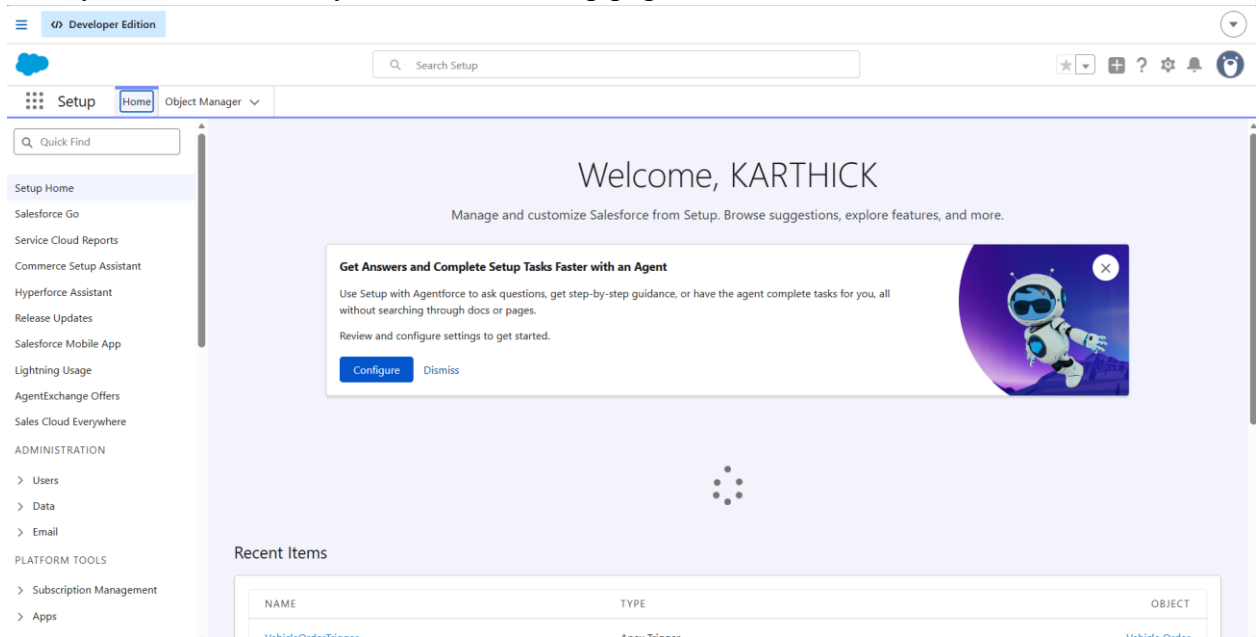
Click on sign me up after filling these.

Activity 2: Account Activation

1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.
2. After Verify the Account, Reset the Password.



3. Give a password and answer a security question and click on change password.
4. Then you will redirect to your salesforce setup page.



MileStone 2 :

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:

Click on gear icon → click setup.

The screenshot displays the Salesforce Setup page for a user named KARTHICK. The interface includes a top navigation bar with the 'Setup' tab selected. A left sidebar contains a 'Quick Find' search bar and a list of setup categories: Setup Home, Salesforce Go, Service Cloud Reports, Commerce Setup Assistant, Hyperforce Assistant, Release Updates, Salesforce Mobile App, Lightning Usage, AgentExchange Offers, Sales Cloud Everywhere, and ADMINISTRATION. The main content area features a welcome message, a section for getting answers and completing setup tasks faster with an agent, and a section for achieving popular business goals. The 'Tableau Next in Slack' goal is highlighted, showing it includes collaboration with Slack and Tableau Next. The 'Cross Cloud' goal is also visible, showing it includes Sales Cloud Everywhere and Agentforce (Default). A 'Setup Menu' dropdown is open on the right, showing options like Setup, Data Cloud Setup, Salesforce Go, Agentforce Vibes, and Developer Console.

Activity 1: Create Vehicle Object

The Vehicle object is created to maintain the important information related to vehicles in the Salesforce application.

To create the Vehicle object:

1. Open **Setup** and go to **Object Manager**.
2. Select **Create** and choose **Custom Object**.

Enter the object label as **Vehicle**.

Set the plural label as **Vehicles**.

Under Record Name, enter **Vehicle Name**.

Select **Text** as the Record Name data type.

The screenshot shows the 'New Custom Object' setup page in Salesforce. The 'Custom Object Information' section is visible, showing the following fields: Label (Vehicle), Plural Label (Vehicles), Object Name (Vehicle), and Description. The 'Enter Record Name Label and Format' section shows the Record Name (Vehicle Name) and Data Type (Text). The 'Optional Features' section has checkboxes for Allow Reports, Allow Activities, Track Field History, Allow in Chatter Groups, and Enable Learning. The 'Object Classification' section has checkboxes for Allow Sharing, Allow Bulk API Access, and Allow Streaming API Access. The 'Deployment Status' section has radio buttons for In Development and Deployed. The 'Search Status' section has a checkbox for Allow Search.

Enable **Allow Reports** and **Allow Search**.

Click **Save**

Activity 2: Create Dealer Object

The Vehicle Dealer object is used to store the details of dealers associated with the vehicles.

To create the Vehicle Dealer object:

1. Open **Setup** and navigate to **Object Manager**.
2. Click **Create** and select **Custom Object**.
3. Enter the label as **Vehicle Dealer**.
4. Set the plural label as **Vehicle Dealers**.
5. For Record Name, enter **Dealer Name**.
6. Select **Text** as the Record Name data type.
7. Enable **Allow Reports** and **Allow Search**.

8. Click **Save**

Activity 3: Creating remaining objects

The remaining custom objects are created to manage customer, order, test drive, and service-related information.

Create the following custom objects using Setup → Object Manager → Create → Custom Object:

1. Vehicle Customer – Record Name: Customer Name
2. Vehicle Order – Record Name: Order Name
3. Vehicle Test Drive – Record Name: Test Drive Name
4. Vehicle Service Request – Record Name: Service Request Name

After entering the required details, enable Allow Reports and Allow Search, then click Save.

Milestone 3 - Tabs

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)

The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with 'tabs' entered. Below the search bar, the 'User Interface' section is expanded, showing 'Rename Tabs and Labels' and 'Tabs'. The 'Tabs' link is highlighted. The main content area is titled 'New Custom Object Tab' and shows 'Step 1 of 3: Enter the Details'. The instructions state: 'Choose the custom object for this new custom tab. Fill in other details.' The form includes a dropdown for 'Object' set to 'None', a 'Tab Style' dropdown, an optional 'Splash Page Custom Link' dropdown set to 'None', and a 'Description' text area. 'Next' and 'Cancel' buttons are at the bottom right.

2. Select Object(Choose your created objects) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.



Activity 2:

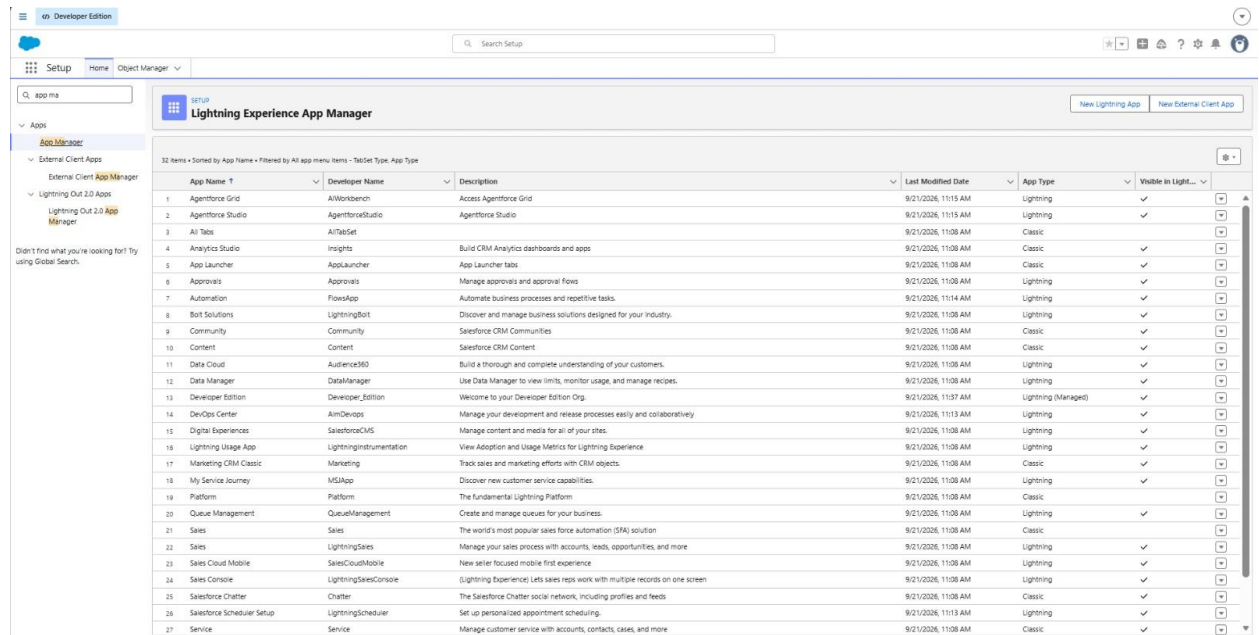
Now create tabs for Other objects as well which we have created above Objects.

Milestone 4- The Lightning App:

Activity 1: Create a Lightning App

To create a lightning app page:

1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



2. Fill the app name in app details and branding as follow
Enter the App Name as: **WhatNext Vision Motor**
Developer Name : this will auto populated
Description : Give a meaningful description
Image : optional (if you want to give any image you can otherwise not mandatory)
Primary color hex value : keep this default
3. Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.

New Lightning App

App Details & Branding

Give your Lightning app a name and description. Upload an image and choose the highlight color for its navigation bar.

App Details

*App Name

*Developer Name

Description

App Branding

Image

Upload

Primary Color Hex Value

#0070C2

Org Theme Options

☐ Use the app's image and color instead of the org's custom theme

App Launcher Preview

Next

4. To Add Navigation Items:

New Lightning App

Navigation Items

Choose the items to include in the app, and arrange the order in which they appear. Users can personalize the navigation to add or move items, but users can't remove or rename the items that you add. Some navigation items are available only for phone or only for desktop. These items are dropped from the navigation bar when the app is viewed in a format that the item doesn't support.

Available Items

Type to filter list...

- AI Prediction Scores
- Accounts
- Action Hub
- Activation Targets
- Activations
- Activity History
- Advertisements (Beta)
- Agent Config
- AgentExchange Go-To-Market App Co...
- AmazonFuturenow Go-To-Market &on Brn

Selected Items

- Vehicle Customers
- Vehicle Dealers
- Vehicle Service Requests
- Vehicle Test Drives
- Vehicles
- Vehicle Orders

Back

Next

Add the required navigation items such as **Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, Vehicle Service Request, Reports, and Dashboard**
Note: select the custom object which we have created in the previous activity.

5. To Add User Profiles:

New Lightning App

User Profiles

Choose the user profiles that can access this app.

Available Profiles

Type to filter list...

- Analytics Cloud Integration User
- Analytics Cloud Security User
- Anypoint Integration
- Authenticated Website
- Authenticated Website
- B2B Reordering Portal Buyer Profile
- Contract Manager
- Custom Marketing Profile
- Custom Sales Profile
- Customize Component Profile

Selected Profiles

- System Administrator

Back

Save & Finish

Keep the remaining settings as default.

Click **Save & Finish**.

Milestone 5 : Fields & Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle Name** – Text
- **Vehicle Model** – Picklist
- **Stock Quantity** – Number
- **Price** – Currency
- **Dealer** – Lookup to Vehicle Dealer
- **Status** – Picklist: Available, Out of Stock, Discontinued

2. Vehicle_Dealer__c (Custom Object)

- **Dealer Name** – Text
- **Dealer Location** – Text
- **Dealer Code** – Auto Number
- **Phone** – Phone
- **Email** – Email

3. Vehicle_Order__c (Custom Object)

- **Vehicle Customer** – Lookup to Vehicle Customer
- **Vehicle** – Lookup to Vehicle
- **Order Date** – Date
- **Status** – Picklist: Pending, Confirmed, Delivered, Canceled

4. Vehicle_Customer__c (Custom Object)

- **Customer Name** – Text
- **Email** – Email
- **Phone** – Phone
- **Address** – Text
- **Preferred Vehicle Type** – Picklist

5. Vehicle_Test_Drive__c (Custom Object)

- **Vehicle Customer** – Lookup to Vehicle Customer
- **Vehicle** – Lookup to Vehicle

- **Test Drive Date** – Date
- **Status** – Picklist: Scheduled, Completed, Canceled

6. Vehicle_Service_Request__c (Custom Object)

- **Vehicle Customer** – Lookup to Vehicle Customer
- **Vehicle** – Lookup to Vehicle
- **Service Date** – Date
- **Issue Description** – Text
- **Status** – **Picklist**: Requested, In Progress, Completed WhatNext Vision Motors

Activity 1: Creating a Field in Vehicle Object

1. Go to Setup → Object Manager → type **Vehicle** in Quick Find → click on the object.
2. Now click on “Fields & Relationships” → New
3. Select Data type as “PickList”.
4. Click on Next
5. Fill the above as following:
 - Field Label: Vehicle Modal
 - Fill PickList Values
 - Click on Next → Next → Save and new.

The screenshot shows the Salesforce Setup interface for creating a new custom field for the 'Vehicle' object. The wizard is at Step 2 of 4, 'Enter the details'. The 'Field Label' is 'Vehicle Modal'. The 'Values' section shows a text area with 'Request' and 'In Progress' entered, separated by a new line. The 'Field Name' is 'Vehicle'. The 'Description' and 'Help Text' fields are empty. The 'Default Value' field is empty. The 'Required' checkbox is checked. The 'Auto add to custom report type' checkbox is checked. The 'Flow Triggers' checkbox is checked. The 'Validation Rules' checkbox is checked. The 'Conditional Field Formatting' checkbox is checked.

Activity 2 : Creating Stock Quantity field in vehicle object

- 1.Go to **Setup → Object Manager → Search Vehicle → Open Vehicle → Fields & Relationships → New**
- 2.Select **Number** as the Data Type
- 3.Click on Next
- 4.**Field Label**, enter **Stock Quality** → **Next** → **Next** → **Save and New**.

Vehicle
New Custom Field

Step 2: Enter the details

Field Label:

Please enter the length of the number and the number of decimal places. For example, a number with a length of 8 and 2 decimal places can accept values up to "12345678.99".

Length: Number of digits to the left of the decimal point

Decimal Places: Number of digits to the right of the decimal point

Field Name:

Description:

Help Text:

☐ Always require a value in this field in order to save a record

☐ Unique

☐ Do not allow duplicate values

☐ External ID

☐ Use this field as the unique record identifier from an external system

☐ AI Prediction

☐ Use this field to store AI prediction scores

☒ Add this field to existing custom report types that contain this entity

Auto add to custom report type

Default Value:

Use formula syntax: Enclose text and literal values in double quotes. "Yes, but" includes numbers without quotes. (2+3)*4 returns an integer (14); 2+3*4 returns the integer in the standard order (14). To reference a field from a Custom Metadata type, use \$CustomMetadata.Type__c and RecordIDName.Field__c.

Activity 3 : Creating a Lookup Relationship in Vehicle Object to Dealer Object.

- 1.Go to **Setup → Object Manager → Vehicle → Fields & Relationships → New**
- 2.Select **Lookup Relationship** as the Data Type.
- 3.Click **Next**.

Vehicle
New Relationship

Step 2: Choose the related object

Select the other object to which this object is related.

Related To:

SETUP > OBJECT MANAGER

Vehicle

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Vehicle

New Relationship

Step 3. Enter the label and name for the lookup field

Step 3 of 6

Field LabelVehicle Dealer

Field NameVehicle_Dealer

Description

Help Text

Child Relationship NameVehicles1

Required☐ Always require a value in this field in order to save a record.

What to do if the lookup record is deleted?☒ Clear the value of this field. You can't choose this option if you make this field required.☐ Delete this record also. This allows users to delete large numbers of records without regard for sharing or visibility constraints.

Field LabelVehicle Dealer

Data TypeLookup

Field NameVehicle_Dealer

Description

Select the profiles to which you want to grant edit access to this field via field-level security. The field will be hidden from all profiles if you do not add it to field-level security.

Field-Level Security for Profile	☒ Visible	☐ Read-Only
Analytics Cloud Integration User	☒	☐
Analytics Cloud Security User	☒	☐
Anypoint Integration	☒	☐
Contract Manager	☒	☐
Cross Org Data Proxy User	☒	☐
Custom: Marketing Profile	☒	☐
Custom: Sales Profile	☒	☐
Custom: Support Profile	☒	☐
Einstein Agent User	☒	☐

4.Select **Dealer** as the **Related To** object.

5.Click **Next** → **Save**

Activity 4 : Creating Remaining Objects and Fields.

MileStone 6 :

Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer's location

Step 1 : Go to **Setup** → **Quick Find** → **Flows**.

Step 2 : Click **Flows**.

Step:3 Click **New Flow**.

New Automation

Get Started with Automations
Select a category, flow type, use search, or let Agentforce build an automation for you.

 **Let Agentforce Help You Build**
Describe what you want to automate and let Agentforce generative AI build a draft flow to get you started.

[Get Started \(Beta\)](#)

Categories

 **Triggered Automations**
Automations launched by records and events. This type of automation runs without user interaction.
[View All Automations](#)

 **Scheduled Automations**
Time-based automations that launch at a specific time or frequency. This type of automation runs without user interaction.
[View All Automations](#)

 **Screen Automations**
Interface-driven automations that guide users through business processes. This type of automation collects or displays...
[View All Automations](#)

 **Autolaunched Automations**
Automations that automatically launch when invoked by APIs, templates, processes, conditions, or something else...
[View All Automations](#)

Frequently Used
[View All Automations](#)

 **Record-Triggered Flow**
Launches when a record is created, updated, or deleted. This autolaunched flow runs in the background.

 **Screen Flow**
Guides users through a business process that's launched from Lightning pages, Experience Cloud sites, quick actions, an...

 **Schedule-Triggered Flow**
Launches at a specified time and frequency for each record in a batch. This autolaunched flow runs in the...

 **Autolaunched Flow (No Trigger)**
Launches when invoked by Apex, processes, REST API, and more. This autolaunched flow runs in the...

Step 4 : Select Record-Triggered Flow

New Automation

**Triggered Automations**
Showing 12 of 97 results

Filter By ☒ Triggered ☐ Scheduled ☐ Screen ☐ Autolaunched

Types (6)

**Record-Triggered Flow**
Launches when a record is created, updated, or deleted. This autolaunched flow runs in the background.

**Platform Event—Triggered Flow**
Launches when a platform event message is received. This autolaunched flow runs in the background.

**Automation Event-Triggered Flow**
Launches when an automation event is received. This type of flow runs in the background without user interaction.

**Activation-Triggered Flow**
Launches when an activation is published. This type of flow runs in the background without user interaction.

**Record-Triggered Orchestration**
Launches when a record is created or updated. An orchestration lets you create a multi-step, multi-user process.

**Record-Triggered Flow Approval Process**
Launches when a record is created or updated. A flow approval process lets you create a multi-step, multi-user approval process.

Templates (6)





[Back](#)

Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :

All Conditions Are Met (AND)

Filed : Status__c

Operator : Equals

Value : Pending

**Record-Triggered Flow**
Start

Object: **Vehicle Order** [Edit](#)

Trigger: **A record is created**

Conditions: 1

Optimize for: **Actions and Related Recor...**

[+ Add Scheduled Paths \(Optional\)](#)

[Open Flow Trigger Explorer for Vehicle...](#)

Run Immediately

Get Customer Information
Get Records

Get Nearest Dealer
Get Records

Assign Dealer to Order

**Configure Start**

Select Object
Select the object whose records trigger the flow when they're created, updated, or deleted.

* Object

Configure Trigger
Trigger the Flow When:
☒ A record is created
☐ A record is updated
☐ A record is created or updated
☐ A record is deleted

Set Entry Conditions
Specify entry conditions to reduce the number of records that trigger the flow and the number of times the flow is executed. Minimizing unnecessary flow executions helps to conserve your org's resources.

Condition Requirements

Field	Operator	Value
Status__c	Equals	Pending

Step 5 : Click + → Select **Get Record**

Step 6 : **Label** : Get Customer Information

Object : Vehicle Customer

Condition :

Field : Id

Operator : Equals

Value : {!\$Record.Vehicle_Customer__c}

The screenshot shows a flow diagram on the left and a 'Get Records' configuration panel on the right. The flow diagram consists of a sequence of steps: 'Run Immediately', 'Get Customer Information' (highlighted with a blue border), 'Get Nearest Dealer', 'Assign Dealer to Order', and 'End'. The 'Get Records' panel is configured with the following details:

- *Label**: Get Customer Information
- *API Name**: Get_Customer_Information
- Description**: (Empty text area)
- Get Records of This Object**:
 - *Object**: Vehicle Customer
- Filter Vehicle Customer Records**:
 - Condition Requirements**: All Conditions Are Met (AND)
 - Field**: Record ID
 - Operator**: Equals
 - Value**: !Record ID
- Sort Vehicle Customer Records**: (Empty section)

Step 7 : Click + → Select **Get Records**

Label : Get Nearest Dealer

Object : Vehicle Dealer

Condition :

Field : Dealer_Location__c

Operator : Equals

Value : {!Get_Customer_Information.Address__c}

The screenshot shows a flow diagram on the left and a 'Get Records' configuration panel on the right. The flow diagram consists of a sequence of steps: 'Run Immediately', 'Get Customer Information', 'Get Nearest Dealer' (highlighted with a blue border), 'Assign Dealer to Order', and 'End'. The 'Get Records' panel is configured with the following details:

- *Label**: Get Nearest Dealer
- *API Name**: Get_Nearest_Dealer
- Description**: (Empty text area)
- Get Records of This Object**:
 - *Object**: Vehicle Dealer
- Filter Vehicle Dealer Records**:
 - Condition Requirements**: All Conditions Are Met (AND)
 - Field**: Dealer Location
 - Operator**: Equals
 - Value**: !Customer Information > Address
- Sort Vehicle Dealer Records**: (Empty section)

Step 8 : Click + → Select Update Records

Label : Assign Dealer to Order

***How to Find Records to Update and Set Their Values :** Use the IDs and all field values from a record or record collection

Select Record(s) to Update : `{!Get_Nearest_Dealer}`

The screenshot shows the Flow Builder interface. On the left, a flow diagram includes steps: 'Open Flow Trigger Explorer for Vehicle...', 'Run Immediately', 'Get Customer Information', 'Get Nearest Dealer', 'Assign Dealer to Order' (highlighted with a blue border), and 'End'. On the right, the 'Update Records' configuration panel is open. It has a 'Label' field with 'Assign Dealer to Order' and an 'API Name' field with 'Assign_Dealer_to_Order'. The 'Description' field is empty. Under 'How to Find Records to Update and Set Their Values', the first option 'Use the vehicle order record that triggered the flow' is selected. The 'Set Filter Conditions' section shows 'Condition Requirements to Update Record' set to 'None—Always Update Record'. The 'Set Field Values for the Vehicle Order Record' section shows a table with one row: 'Vehicle Dealer' mapped to 'le Dealer from Get Nearest Dealer > Record ID'.

Step 9 : Click Save and Give label Name and Activate Flow.

Label Name : Auto Assign Dealer

Save as

Save As

A New Version

* Flow Label

Assign Nearest Dealer to Vehicle Order

* Flow API Name

Assign_Nearest_Dealer_to_Vehicle_Order

Description

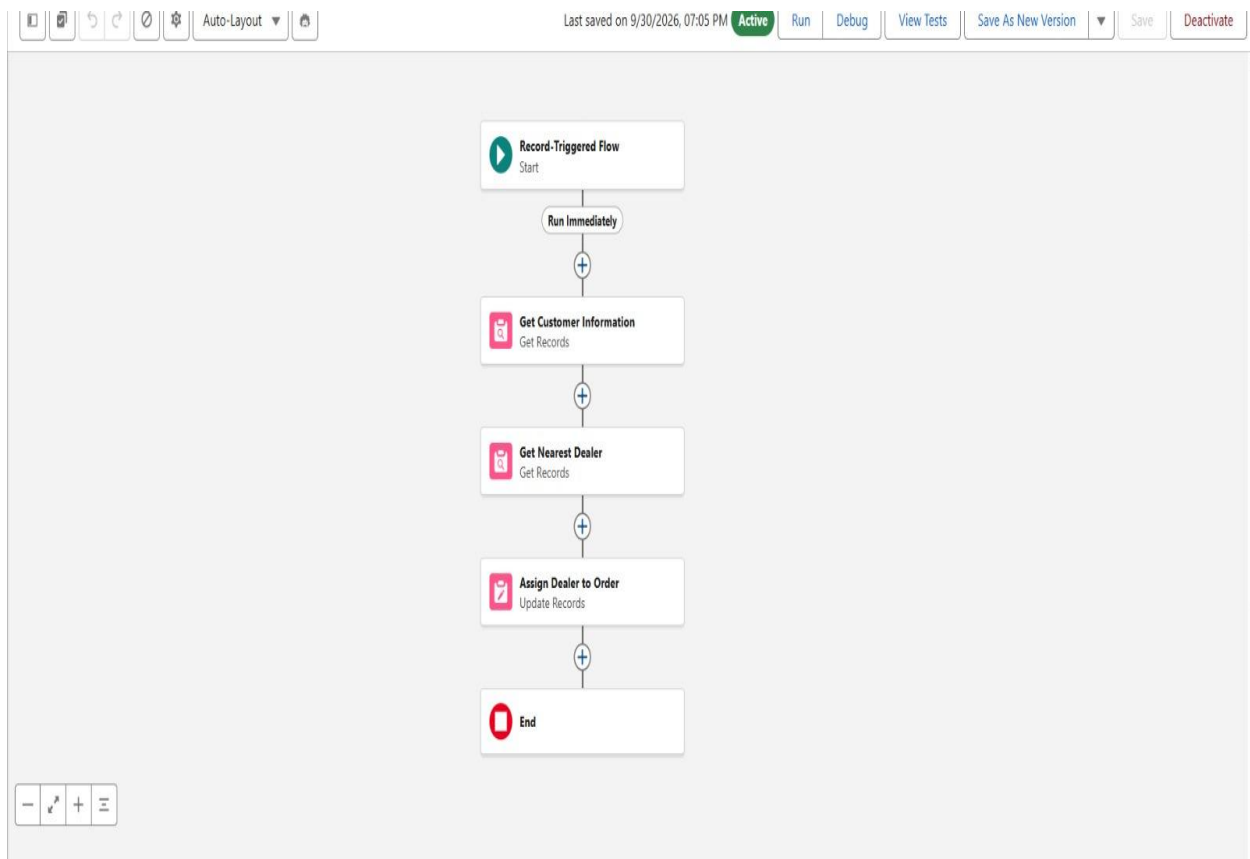
Automatically assigns the nearest vehicle dealer to a newly created vehicle order based on the customer's location.

Show Advanced

Cancel

Save

Step 10 : Activate Flow



Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow

Step 2 : Select Vehicle Test Drive Object

Trigger the Flow When : A record is created or updated

Set Entry Condition :

All Conditions Are Met (AND)

Filed : Status__c

Operator : Equals

Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

Label: Reminder Before Test Drive.

Time Source: Test_Drive_Date__c

Offset Number: 1.

Offset Options: Days Before.

Click Done.

Configure Scheduled Paths

SCHEDULED PATHS

+

Run Immediately

Reminder Before Test Drive

Mark Test Drive Completed

Add a scheduled path if you want a flow path to run a certain amount of time after the triggering record is created or updated. That time can be based on the triggering event or on a specified date or date/time field in the record. A separate flow interview runs for each scheduled path. Each scheduled path interview is queued to run at the scheduled time, but can be delayed until system resources become available. Path labels appear on connectors in the canvas.

SCHEDULED PATH DETAILS

* Path Label

Reminder Before Test Drive

* API Name ⓘ

Reminder_Before_Test_Drive

* Time Source

Vehicle_Test_Drive__c: Test Drive Date

* Offset Number

1

* Offset Options ⓘ

Days Before

> Show advanced options

Delete Path

Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!\$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

 Get Records ×

* Label

Get Customer Information - Reminder

* API Name 

Get_Customer_Information_Reminder

Description

Get Records of This Object

* Object

Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements

All Conditions Are Met (AND) ▼

Field	Operator	Value
Aa Record ID ×	Equals ▼	Aa ...ehicle Customer > Record ID × 

+ Add Condition

Sort Vehicle Customer Records

Sort Order

Step 5: Send Reminder Email

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

 Send Email ×

* Label

* API Name 

Description

Use values from earlier in the flow to set the inputs for the "Send Email" core action. To use its outputs later in the flow, store them in variables.

Set Input Values

✓ Configure Recipient Details

A_a Recipient Addresses 



A_a ...et Customer Information - Reminder > Email ×

A_a CC Addresses 



8 . Api Name : EmailSent

* API Name ⓘ

EmailSend

Description

Test Drive Reminder Email

* Body ⓘ

Insert a resource... 🔍

View as Rich Text ▼

Dear Customer,

This is a reminder that your test drive is scheduled for tomorrow.

If you need to reschedule, please contact WhatNext Vision Motors.

Thank you,
WhatNext Vision Motors Team

Salesforce Sans ▼

13 ▼

■ ▼

B

I

U

:≡

½≡

→≡

+≡

≡

≡

≡

🔗

🖼️

ℹ️

Label Name : Test Drive Reminder

Save As

A New Version

* Flow Label

Vehicle Test Drive Reminder

* Flow API Name ⓘ

Vehicle_Test_Drive_Reminder

Description

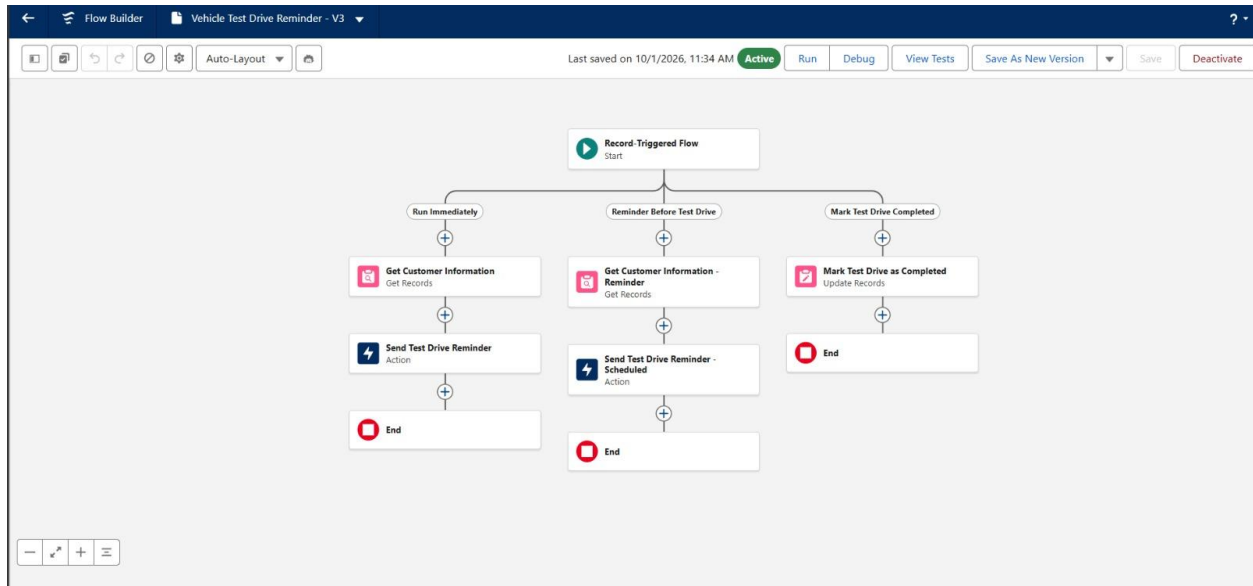
Show Advanced

Cancel

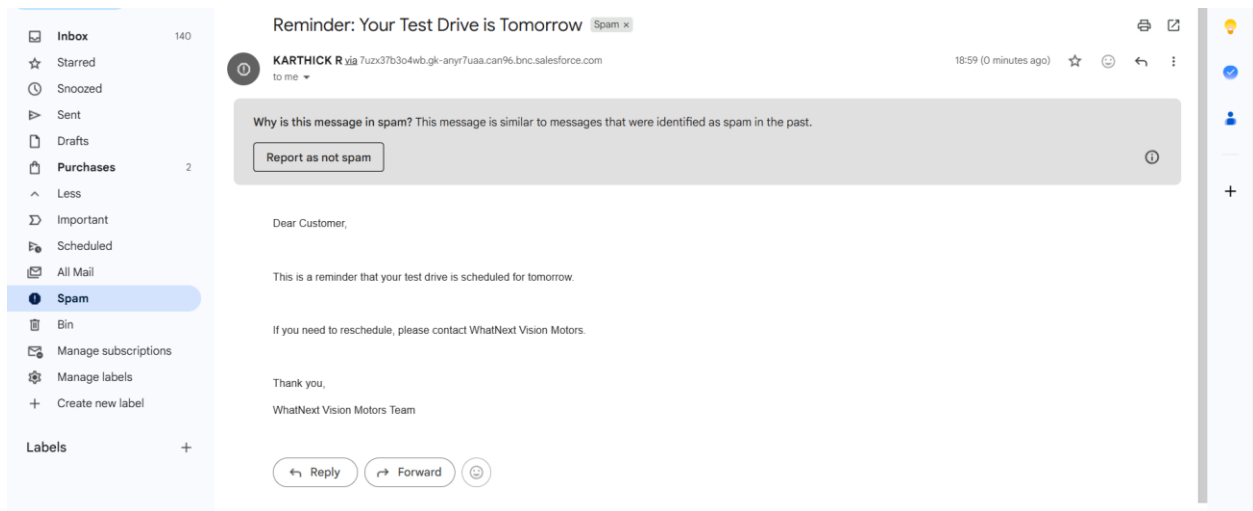
Save

Step 7 : Activate Flow

Flow Page After Creation :



Test your Flow:





Send Email



Configure Recipient Details

Recipient Addresses ⓘ

Enter value or search resources...



...le_Test_Drive_c > Vehicle Customer > Email X

CC Addresses ⓘ

Enter value or search resources...



BCC Addresses ⓘ

Enter value or search resources...



Configure Sender Details

Sender Type ⓘ

Aa ▾

CurrentUser ▾

Sender Email Address ⓘ

Aa ▾

Search records...



Configure Email Content

Compose Email Content

Set email details and compose message.

Use Email Template

Select an existing email template.

Subject ⓘ



Reminder: Your Test Drive is Tomorrow



Body ⓘ



EmailSend X

Rich-Text-Formatted Body ⓘ

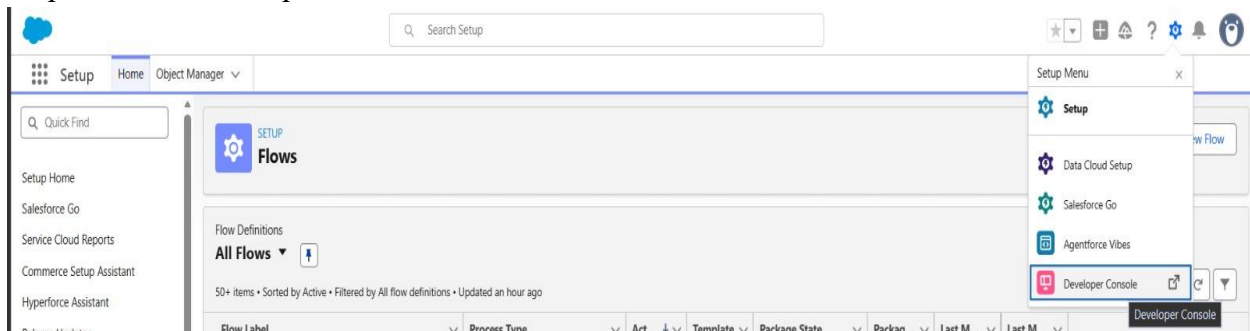


True X

MileStone 7 :

Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon

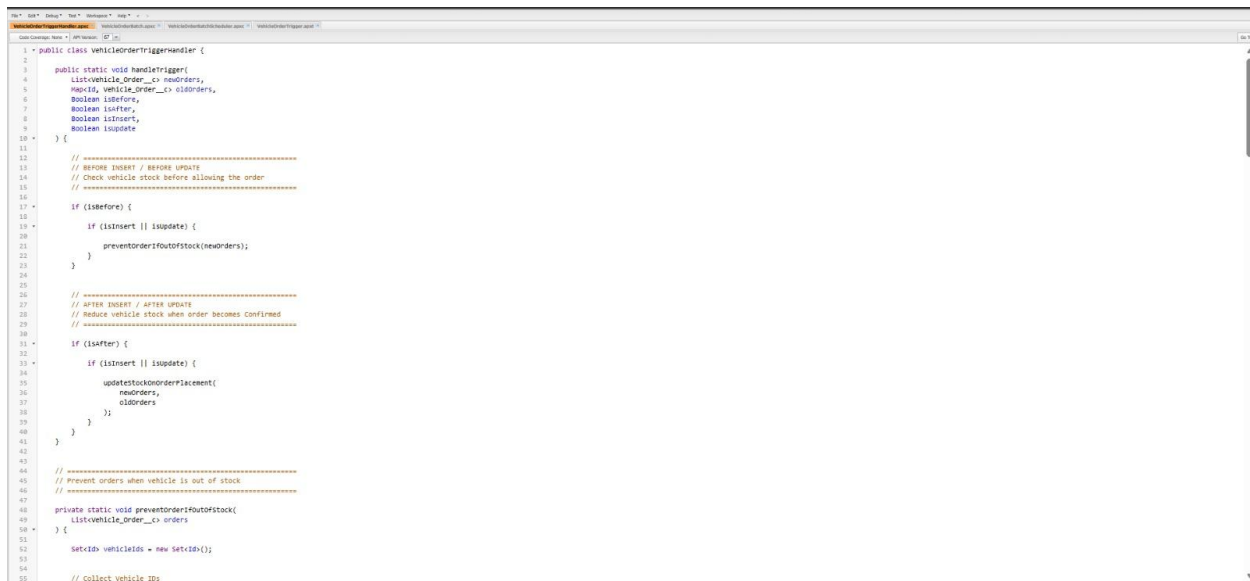


Step 2 : click File then Select New then Select Apex.

Step 3 : Give Name For Apex Class

Class Name : VehicleOrderTriggerhandler

Step 4 : Write Apex Code



```
154 * For (vehicle_order_c order : orders) {
155 *
156 *     if (order.vehicle_c != null) {
157 *         vehicleIds.add(order.vehicle_c);
158 *     }
159 *
160 *     if (!vehicleIds.isEmpty()) {
161 *         Map<Id, vehicle_c> vehicleStockMap =
162 *             new Map<>();
163 *
164 *         // get vehicle stock
165 *         for (vehicle_c vehicle : I
166 *             SELECT Id, stock_quantity_c
167 *             FROM vehicle_c
168 *             WHERE Id IN :vehicleIds
169 *         ) {
170 *             vehicleStockMap.put(
171 *                 vehicle.Id,
172 *                 vehicle
173 *             );
174 *         }
175 *
176 *         // Check stock
177 *         for (vehicle_order_c order : orders) {
178 *             if (
179 *                 order.vehicle_c != null &&
180 *                 vehicleStockMap.containsKey(order.vehicle_c)
181 *             ) {
182 *                 vehicle_c vehicle =
183 *                     vehicleStockMap.get(order.vehicle_c);
184 *
185 *                 if (
186 *                     vehicle.stock_quantity_c == null ||
187 *                     vehicle.stock_quantity_c <= 0
188 *                 ) {
189 *                     order.addError(
190 *                         "This vehicle is out of stock. Order cannot be placed."
191 *                     );
192 *                 }
193 *             }
194 *         }
195 *     }
196 * }
197 * }
```

```
111 // =====
112 // Update vehicle stock when order becomes Confirmed
113 // =====
114 private static void updateStockOnOrderPlacement(
115     List<vehicle_order_c> orders,
116     Map<Id, vehicle_order_c> oldOrders
117 ) {
118     Set<Id> vehicleIds = new Set<>();
119
120     // =====
121     // Find orders that have JUST become Confirmed
122     // =====
123     for (vehicle_order_c order : orders) {
124         Boolean becameConfirmed = false;
125
126         // INSERT
127         if (
128             oldOrders == null ||
129             !oldOrders.containsKey(order.Id)
130         ) {
131             if (order.status_c == 'Confirmed') {
132                 becameConfirmed = true;
133             }
134         }
135
136         // UPDATE
137         else {
138             vehicle_order_c oldOrder =
139                 oldOrders.get(order.Id);
140
141             if (
142                 order.status_c == 'Confirmed' &&
143                 oldOrder.status_c != 'Confirmed'
144             ) {
145                 becameConfirmed = true;
146             }
147         }
148
149         // Collect vehicle Id
150         if (
151             becameConfirmed &&
152         ) {
153         }
154     }
155 }
```

```
165 * } { order.vehicle_c != null
166 * } {
167 *     vehicles.add(order.vehicle_c);
168 * }
169 * }
170 * }
171 * }
172 * }
173 * }
174 * }
175 * }
176 * }
177 * }
178 * }
179 * }
180 * }
181 * }
182 * }
183 * }
184 * }
185 * }
186 * }
187 * }
188 * }
189 * }
190 * }
191 * }
192 * }
193 * }
194 * }
195 * }
196 * }
197 * }
198 * }
199 * }
200 * }
201 * }
202 * }
203 * }
204 * }
205 * }
206 * }
207 * }
208 * }
209 * }
210 * }
211 * }
212 * }
213 * }
214 * }
215 * }
216 * }
217 * }
218 * }
219 * }
220 * }
```

```
221 * }
222 * }
223 * }
224 * }
225 * }
226 * }
227 * }
228 * }
229 * }
230 * }
231 * }
232 * }
233 * }
234 * }
235 * }
236 * }
237 * }
238 * }
239 * }
240 * }
241 * }
242 * }
243 * }
244 * }
245 * }
246 * }
247 * }
248 * }
249 * }
250 * }
251 * }
252 * }
253 * }
254 * }
255 * }
256 * }
257 * }
258 * }
259 * }
260 * }
261 * }
262 * }
263 * }
264 * }
265 * }
266 * }
267 * }
268 * }
269 * }
270 * }
271 * }
272 * }
273 * }
274 * }
275 * }
```

```
276 * }
277 * }
278 * }
279 * }
280 * }
281 * }
282 * }
283 * }
284 * }
285 * }
286 * }
```

```

public class VehicleOrderTriggerHandler {

    public static void handleTrigger(
        List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldOrders,
        Boolean isBefore,
        Boolean isAfter,
        Boolean isInsert,
        Boolean isUpdate
    ) {

        // =====
        // BEFORE INSERT / BEFORE UPDATE
        // Check vehicle stock before allowing the order
        // =====

        if (isBefore) {

            if (isInsert || isUpdate) {

                preventOrderIfOutOfStock(newOrders);
            }
        }

        // =====
        // AFTER INSERT / AFTER UPDATE
        // Reduce vehicle stock when order becomes Confirmed
        // =====

        if (isAfter) {

            if (isInsert || isUpdate) {

                updateStockOnOrderPlacement(
                    newOrders,
                    oldOrders
                );
            }
        }
    }

    // =====

```

```

// Prevent orders when vehicle is out of stock
// =====

private static void preventOrderIfOutOfStock(
    List<Vehicle_Order__c> orders
) {

    Set<Id> vehicleIds = new Set<Id>();

    // Collect Vehicle IDs
    for (Vehicle_Order__c order : orders) {

        if (order.Vehicle__c != null) {

            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {

        Map<Id, Vehicle__c> vehicleStockMap =
            new Map<Id, Vehicle__c>();

        // Get vehicle stock
        for (Vehicle__c vehicle : [
            SELECT Id, Stock_Quantity__c
            FROM Vehicle__c
            WHERE Id IN :vehicleIds
        ]) {

            vehicleStockMap.put(
                vehicle.Id,
                vehicle
            );
        }

        // Check stock
        for (Vehicle_Order__c order : orders) {

            if (

```

```

        order.Vehicle__c != null &&
        vehicleStockMap.containsKey(order.Vehicle__c)
    ) {

        Vehicle__c vehicle =
            vehicleStockMap.get(order.Vehicle__c);

        if (
            vehicle.Stock_Quantity__c == null ||
            vehicle.Stock_Quantity__c <= 0
        ) {

            order.addError(
                'This vehicle is out of stock. Order cannot be placed.'
            );
        }
    }
}

// =====
// Update vehicle stock when order becomes Confirmed
// =====

private static void updateStockOnOrderPlacement(
    List<Vehicle_Order__c> orders,
    Map<Id, Vehicle_Order__c> oldOrders
) {

    Set<Id> vehicleIds = new Set<Id>();

    // -----
    // Find orders that have JUST become Confirmed
    // -----

    for (Vehicle_Order__c order : orders) {

        Boolean becameConfirmed = false;

```

```

// INSERT
if (
    oldOrders == null ||
    !oldOrders.containsKey(order.Id)
) {

    if (order.Status__c == 'Confirmed') {

        becameConfirmed = true;
    }
}

// UPDATE
else {

    Vehicle_Order__c oldOrder =
        oldOrders.get(order.Id);

    if (
        order.Status__c == 'Confirmed' &&
        oldOrder.Status__c != 'Confirmed'
    ) {

        becameConfirmed = true;
    }
}

// Collect Vehicle ID
if (
    becameConfirmed &&
    order.Vehicle__c != null
) {

    vehicleIds.add(order.Vehicle__c);
}

// -----
// Get vehicles
// -----

```

```

if (!vehicleIds.isEmpty()) {

    Map<Id, Vehicle__c> vehicleStockMap =
        new Map<Id, Vehicle__c>();

    for (Vehicle__c vehicle : [
        SELECT Id, Stock_Quantity__c
        FROM Vehicle__c
        WHERE Id IN :vehicleIds
    ]) {

        vehicleStockMap.put(
            vehicle.Id,
            vehicle
        );
    }

    // -----
    // IMPORTANT:
    // Use Map instead of List.
    // This prevents Duplicate ID errors.
    // -----

    Map<Id, Vehicle__c> vehiclesToUpdate =
        new Map<Id, Vehicle__c>();

    // -----
    // Reduce stock
    // -----

    for (Vehicle_Order__c order : orders) {

        Boolean becameConfirmed = false;

        // INSERT
        if (
            oldOrders == null ||
            !oldOrders.containsKey(order.Id)
        ) {

```

```

        if (order.Status__c == 'Confirmed') {

            becameConfirmed = true;
        }
    }

// UPDATE
else {

    Vehicle_Order__c oldOrder =
        oldOrders.get(order.Id);

    if (
        order.Status__c == 'Confirmed' &&
        oldOrder.Status__c != 'Confirmed'
    ) {

        becameConfirmed = true;
    }
}

// Update vehicle stock
if (
    becameConfirmed &&
    order.Vehicle__c != null &&
    vehicleStockMap.containsKey(order.Vehicle__c)
) {

    Vehicle__c vehicle =
        vehicleStockMap.get(order.Vehicle__c);

    if (
        vehicle.Stock_Quantity__c != null &&
        vehicle.Stock_Quantity__c > 0
    ) {

        vehicle.Stock_Quantity__c -= 1;
    }
}

```

```

        // Store using Vehicle ID.
        // Duplicate IDs are prevented.
        vehiclesToUpdate.put(
            vehicle.Id,
            vehicle
        );
    }
}

// -----
// Update vehicles
// -----

if (!vehiclesToUpdate.isEmpty()) {
    update vehiclesToUpdate.values();
}
}
}
}

```

Step 7 : Call Apex Class in Trigger Class

```

1  class VehicleOrderSearch implements SearchableSubject {
2
3      global database.executeQuery start(Database.SearchContext bc) {
4
5          return database.executeQuery {
6              SELECT ID, Status_1, vehicle_1
7              FROM Vehicle_Order_1
8              WHERE Status_1 = "Pending"
9          }
10     }
11
12     global void execute(
13         Database.SearchContext bc,
14         List<Vehicle_Order_1> orderList
15     ) {
16         Set<Id> vehicleIds = new Set<Id>();
17
18         // Collect vehicle IDs
19         for (Vehicle_Order_1 order : orderList) {
20             if (order.vehicle_1 != null) {
21                 vehicleIds.add(order.vehicle_1);
22             }
23         }
24
25         if (vehicleIds.isEmpty()) {
26             return;
27         }
28
29         // Get vehicles
30         Map<Id, Vehicle_1> vehicleMap = new Map<Id, Vehicle_1>();
31         {
32             SELECT ID, Stock_Quantity_1
33             FROM Vehicle_1
34             WHERE ID IN :vehicleIds
35         }
36     }
37
38     // Map present duplicate IDs
39     Map<Id, Vehicle_Order_1> orderMap =
40         new Map<Id, Vehicle_Order_1>();
41
42     Map<Id, Vehicle_1> vehicleMap =
43         new Map<Id, Vehicle_1>();
44
45     // Process orders
46     for (Vehicle_Order_1 order : orderList) {
47
48         if (order.vehicle_1 == null) {
49             continue;
50         }
51
52         if (vehicleMap.containsKey(order.vehicle_1)) {
53             continue;
54         }
55     }
56 }

```

```
86
87
88     vehicle__c vehicle = vehicleMap.get(order.vehicle__c);
89
90     if (vehicle.stock_quantity__c != null &&
91         vehicle.stock_quantity__c > 0) {
92
93         order.status__c = 'Confirmed';
94
95         vehicle.stock_quantity__c =
96             vehicle.stock_quantity__c - 1;
97
98         ordersToUpdate.put(order.id, order);
99         vehiclesToUpdate.put(vehicle.id, vehicle);
100     }
101 }
102
103 // update orders
104 if (!ordersToUpdate.isEmpty()) {
105     update ordersToUpdate.values();
106 }
107
108 // update vehicles
109 if (!vehiclesToUpdate.isEmpty()) {
110     update vehiclesToUpdate.values();
111 }
112 }
113
114 global void finish(Database.BatchableContext bc) {
115     System.debug('vehicle order batch job completed.');
```

```
86
87
88     vehicle__c vehicle = vehicleMap.get(order.vehicle__c);
89
90     if (vehicle.stock_quantity__c != null &&
91         vehicle.stock_quantity__c > 0) {
92
93         order.status__c = 'Confirmed';
94
95         vehicle.stock_quantity__c =
96             vehicle.stock_quantity__c - 1;
97
98         ordersToUpdate.put(order.id, order);
99         vehiclesToUpdate.put(vehicle.id, vehicle);
100     }
101 }
102
103 // update orders
104 if (!ordersToUpdate.isEmpty()) {
105     update ordersToUpdate.values();
106 }
107
108 // update vehicles
109 if (!vehiclesToUpdate.isEmpty()) {
110     update vehiclesToUpdate.values();
111 }
112 }
113
114 global void finish(Database.BatchableContext bc) {
115     System.debug('vehicle order batch job completed.');
```

Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class

```
File • Edit • Debug • Test • Workspace • Help • < >
VehicleOrderTriggerHandler.apex • VehicleOrderBatch.apex • VehicleOrderBatchScheduler.apex • VehicleOrderTrigger.apex
Code Coverage: None • API Version: 67 • Go
1 global class VehicleOrderBatchScheduler implements Schedulable {
2
3     global void execute(SchedulableContext sc) {
4
5         VehicleOrderBatch batchJob =
6             new VehicleOrderBatch();
7
8         Database.executeBatch(batchJob, 50);
9     }
10 }
```

Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {  
  
    global void execute(SchedulableContext sc) {  
  
        VehicleOrderBatch batchJob =  
            new VehicleOrderBatch();  
  
        Database.executeBatch(batchJob, 50);  
    }  
}
```

Step 10 : Schedule the Batch Job To run the batch job every night at midnight :



Enter Apex Code

1

String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM

2

System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());

☐ Open Log

Execute

Execute Highlighted

Source Code :

String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM

System.schedule('Daily Vehicle Order Processing', cronExp, new
VehicleOrderBatchScheduler());

Step 11 : You Can Check where this Schedule Job is Running

Q job

Environments

Jobs

Apex Flex Queue

Apex Jobs

Background Jobs

Bulk Data Load Jobs

Scheduled Jobs

Didn't find what you're looking for? Try using Global Search.

SETUP

Scheduled Jobs

Help for this Page

All Scheduled Jobs

The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.

Percentage of Scheduled Jobs Used: 2%

You have currently used 2 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: All Scheduled Jobs Create New View

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other All

Schedule Apex

Action	Job Name *	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Inventory Sync	B_KARTHICK	9/30/2026, 11:19 AM	10/1/2026, 12:00 PM	10/2/2026, 12:00 PM	Scheduled Apex	08egK00000nykDj
Manage Del Pause Job	Daily Vehicle Order Processing	B_KARTHICK	9/30/2026, 11:21 AM	10/1/2026, 12:00 PM	10/2/2026, 12:00 PM	Scheduled Apex	08egK00000hxDJS
Del	Metalytics Data Loader Job for Org : 00DgK000000anyr7	User Integration	9/21/2026, 11:13 AM	10/1/2026, 3:58 PM	10/2/2026, 3:58 PM	Autonomous Data Loader Job	08egK000000gnu2c
	Program Milestone Computation Cron Job	Process, Automated	9/21/2026, 11:13 AM	10/1/2026, 4:59 PM	10/1/2026, 11:59 PM	Program Milestone Computation Cron Job	08egK000000gnu2a
	Program Status Update Cron Job	Process, Automated	9/21/2026, 11:13 AM	10/1/2026, 8:00 PM	10/2/2026, 5:00 AM	Program Status Update Cron Job	08egK000000gnu2b

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other All

Thank you